

Chapter 29: Recursion in Java

Personal Notes

Recursion is one of the most powerful — and initially most confusing — ideas in programming. A function that calls itself? It sounds like a bug. But once it clicks, recursion becomes the natural way to solve a whole class of problems: trees, graphs, divide-and-conquer, backtracking, dynamic programming.

This is also where DSA interviews lean heavily. At Google and similar companies, you'll be asked recursive problems in nearly every coding round. Master this chapter and the entire next level of DSA — trees, graphs, recursion-based DP — becomes approachable.

1. What is Recursion?

Recursion is when a function CALLS ITSELF to solve a smaller version of the same problem. Each call works on a slightly simpler input until the problem becomes trivial.

The Core Idea

Every recursive solution has TWO PARTS:

- BASE CASE — the simplest version of the problem, where you can return an answer directly without recursing
- RECURSIVE CASE — break the problem into a smaller version, call the function on that smaller version, and combine the result

```
public int factorial(int n) {  
    if (n <= 1) return 1;           ← BASE CASE  
    return n * factorial(n - 1);    ← RECURSIVE CASE  
}
```

Mental model: Think of recursion as DELEGATION. "I don't know how to solve factorial(5), but if someone gives me factorial(4), I can multiply it by 5." You delegate to a smaller version of yourself, and trust that it'll come back with the answer.

2. The Classic Example — Factorial

Factorial of n (written $n!$) is $n \times (n-1) \times (n-2) \times \dots \times 1$. It's the perfect first recursion because the math itself is naturally recursive.

Mathematical Definition

$n! = n \times (n-1)!$	(recursive)
$0! = 1$	(base case)

Java Implementation

```
public class Main {  
    public static int factorial(int n) {  
        // Base case  
        if (n <= 1) return 1;  
  
        // Recursive case  
        return n * factorial(n - 1);  
    }  
  
    public static void main(String[] args) {  
        System.out.println(factorial(5));    // 120  
    }  
}
```

How factorial(4) Actually Runs

```
factorial(4)  
  = 4 * factorial(3)  
    = 3 * factorial(2)  
      = 2 * factorial(1)          ← base case returns  
        = 1  
      = 2 * 1 = 2  
    = 3 * 2 = 6  
  = 4 * 6 = 24
```

Result: 24

Each call WAITS for the next one to finish, then multiplies its number into the returned value. The deepest call hits the base case first and "unwinds" outward, building up the final answer.

3. The Call Stack — How Recursion Really Works

Behind the scenes, Java uses the CALL STACK to remember where each function call left off. Every active function call has a STACK FRAME holding its local variables and where to return.

Stack During factorial(4)

Time → →

Top of stack	factorial(1)	factorial(1) returns 1
	+-----+	→
	factorial(2)	factorial(2) returns 2
	+-----+	→
	factorial(3)	factorial(3) returns 6
	+-----+	→
	factorial(4)	factorial(4) returns 24
	+-----+	
	main	

+-----+

Each new recursive call PUSHES a new frame onto the stack. When the base case returns, the stack starts UNWINDING — frames pop off one by one as each call computes its result.

Memory cost of recursion: Each recursive call uses stack memory. If you recurse 100,000 times, you have 100,000 frames stacked up. Stack space is limited (~512 KB on most JVMs) — go too deep and you crash with `StackOverflowError`. This is why deeply recursive algorithms often hit limits that iterative ones don't.

4. Three Steps to Write Any Recursive Function

When you're stuck, run through these three questions:

1. Identify the BASE CASE — what's the smallest possible input, and what should you return for it?
2. Define the RECURSIVE CASE — how does the problem at size n relate to the problem at size $n-1$?
3. Make sure progress happens — every recursive call must MOVE TOWARD the base case (smaller n , shorter string, fewer items)

Example: Sum from 1 to N

```
public static int sum(int n) {  
    if (n == 0) return 0;           // base case  
    return n + sum(n - 1);         // recursive case  
}
```

```
sum(4) → 4 + sum(3)  
       → 4 + 3 + sum(2)  
           → 4 + 3 + 2 + sum(1)  
               → 4 + 3 + 2 + 1 + sum(0)  
                   → 4 + 3 + 2 + 1 + 0  
                       → 4 + 3 + 2 + 1 = 10
```

5. More Classic Examples

5.1 Sum of Digits

```
public static int sumOfDigits(int n) {  
    if (n == 0) return 0;  
    return (n % 10) + sumOfDigits(n / 10);  
}
```

```
sumOfDigits(1234)  
= 4 + sumOfDigits(123)  
= 3 + sumOfDigits(12)
```

```

        = 2 + sumOfDigits(1)
          = 1 + sumOfDigits(0)
            = 0
    = 1 + 2 + 3 + 4 = 10

```

5.2 Power (a^b)

```

public static int power(int a, int b) {
    if (b == 0) return 1;
    return a * power(a, b - 1);
}

power(2, 5) = 2 * power(2, 4) = 2 * 16 = 32

```

5.3 GCD (Greatest Common Divisor)

Uses Euclid's algorithm: $\text{GCD}(a, b) = \text{GCD}(b, a \% b)$. The recursion shrinks the problem extremely quickly.

```

public static int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

gcd(48, 18) = gcd(18, 12) = gcd(12, 6) = gcd(6, 0) = 6

```

5.4 Print 1 to N (Without a Loop)

```

public static void print1ToN(int n) {
    if (n == 0) return;           // base case
    print1ToN(n - 1);             // first recurse (so smaller prints
    first)
    System.out.print(n + " ");    // then print
}

print1ToN(5);    // prints: 1 2 3 4 5

```

Subtle point — WHEN you do the work matters: If you swap the two lines (print first, then recurse), you'll get "5 4 3 2 1" — printed BEFORE going deeper. Recursion lets you do work on the way DOWN (forward) or on the way BACK UP (in reverse). This trick is the foundation of tree traversals.

6. Recursion on Arrays

6.1 Sum of Array Elements

```

public static int sumArray(int[] arr, int i) {
    if (i == arr.length) return 0;
    return arr[i] + sumArray(arr, i + 1);
}

```

```
int[] nums = {1, 2, 3, 4, 5};
sumArray(nums, 0);    // 15
```

6.2 Find Maximum Element

```
public static int findMax(int[] arr, int i) {
    if (i == arr.length - 1) return arr[i];
    int restMax = findMax(arr, i + 1);
    return Math.max(arr[i], restMax);
}
```

```
int[] nums = {3, 1, 9, 5, 7};
findMax(nums, 0);    // 9
```

6.3 Linear Search

```
public static int search(int[] arr, int target, int i) {
    if (i == arr.length) return -1;
    if (arr[i] == target) return i;
    return search(arr, target, i + 1);
}
```

```
int[] nums = {10, 20, 30, 40};
search(nums, 30, 0);    // 2
```

7. Recursion on Strings

7.1 Reverse a String

```
public static String reverse(String s) {
    if (s.length() <= 1) return s;
    return reverse(s.substring(1)) + s.charAt(0);
}
```

```
reverse("hello")
= reverse("ello") + "h"
  = reverse("llo") + "e" + "h"
    = reverse("lo") + "l" + "e" + "h"
      = reverse("o") + "l" + "l" + "e" + "h"
        = "o"
          = "ol" + "l" + "e" + "h" = "olle" + "h"
            = "olleh"
```

7.2 Check Palindrome

```
public static boolean isPalindrome(String s, int left, int right) {
    if (left >= right) return true;    // base - done
```

```

    if (s.charAt(left) != s.charAt(right))
        return false; // mismatch
    return isPalindrome(s, left + 1, right - 1);
}

isPalindrome("madam", 0, 4); // true
isPalindrome("hello", 0, 4); // false

```

Two-pointer + recursion: This is your two-pointer technique from array practice (Set 2), but expressed recursively. Either form works — recursion is sometimes easier to reason about because each call handles exactly ONE comparison.

8. Fibonacci — Multiple Recursive Calls

Up to now, each function called itself once. Fibonacci calls itself TWICE per step — this is where recursion starts to look like a tree.

The Math

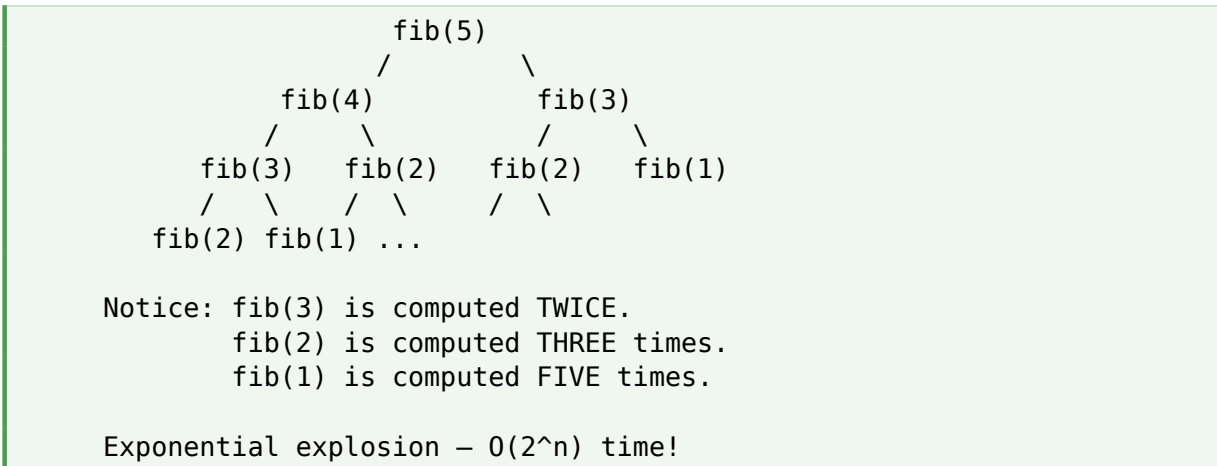
```
fib(0) = 0
fib(1) = 1
fib(n) = fib(n-1) + fib(n-2)
```

The Code

```
public static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

fib(5) = 5
```

The Recursion Tree for fib(5)



Notice: fib(3) is computed TWICE.
fib(2) is computed THREE times.
fib(1) is computed FIVE times.

Exponential explosion – $O(2^n)$ time!

Important: Naive recursive Fibonacci is BEAUTIFUL but TERRIBLE for performance. fib(40) takes seconds. fib(50) hangs. This is the classic motivation for MEMOIZATION and DYNAMIC PROGRAMMING — caching results so each subproblem is solved only once. We'll see this in the DP chapter.

Memoized Fibonacci (Preview of DP)

```
import java.util.HashMap;
import java.util.Map;

static Map<Integer, Integer> memo = new HashMap<>();

public static int fib(int n) {
    if (n <= 1) return n;
    if (memo.containsKey(n)) return memo.get(n);
    int result = fib(n - 1) + fib(n - 2);
    memo.put(n, result);
    return result;
}

// Now O(n) time. fib(50) is instant.
```

9. Recursion vs Iteration

Most recursive problems can also be solved iteratively (with loops). When should you use which?

Aspect	Recursion	Iteration
Code clarity	Often cleaner for tree-like problems	Often cleaner for simple linear work
Memory	Uses stack — limited depth	Uses constant stack space
Speed	Slight function-call overhead	Usually faster
Best for	Trees, graphs, divide & conquer, DP	Counting, accumulation, simple traversal
Risk	StackOverflowError for deep recursion	Easier to get loops wrong (off-by-one)

Rule of thumb: If the problem has natural "smaller versions" (subtrees, sub-arrays, sub-strings), recursion is usually cleaner. If it's a flat linear traversal (sum, max, count), iteration is usually simpler and faster. For interviews, BOTH are good to know — you'll often be asked to convert between them.

10. Time & Space Complexity of Recursion

Two things to think about:

10.1 Time Complexity

How many TOTAL function calls are made? That's your time complexity.

- $\text{factorial}(n)$ — n calls $\rightarrow O(n)$ time
- $\text{sum}(n)$ — n calls $\rightarrow O(n)$ time
- $\text{gcd}(a, b)$ — $\log(\min(a, b))$ calls $\rightarrow O(\log n)$ time
- $\text{fib}(n)$ — about 2^n calls $\rightarrow O(2^n)$ time (exponential!)
- Binary search recursive — $O(\log n)$

10.2 Space Complexity

How DEEP does the call stack get at its worst? That's your space complexity.

- $\text{factorial}(n)$ — stack depth $n \rightarrow O(n)$ space
- $\text{fib}(n)$ — at any moment, stack depth is at most $n \rightarrow O(n)$ space (NOT 2^n !)
- Binary search recursive — $O(\log n)$ space

Subtle but important: $\text{fib}(n)$ makes 2^n CALLS but the stack is never more than n deep at any one time (depth-first). So time is $O(2^n)$ but space is $O(n)$. Distinguishing "total calls" from "deepest stack" is a common interview question.

11. Tail Recursion

A recursion is TAIL RECURSIVE when the recursive call is the LAST thing the function does — there's nothing left to do with the return value.

Not Tail Recursive (Classic Factorial)

```
public static int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);    // multiply AFTER the recursive call
}
```

Tail Recursive Version

```
public static int factorial(int n, int acc) {
    if (n <= 1) return acc;
    return factorial(n - 1, acc * n);    // last thing – pure recursive call
}

// Call with: factorial(5, 1)
```


In tail recursion, the function passes the running result as an extra parameter (accumulator). Once the base case is reached, the answer is already built up.

Java does NOT optimize tail recursion: Languages like Scala and Kotlin can collapse tail recursion into loops (no stack growth). Java does NOT — every tail call still pushes a stack frame. So in Java, tail recursion is mainly a mental model exercise, not a real optimization. For deep work, use loops or convert to iteration.

12. Common Mistakes

Mistake 1: Missing Base Case

```
public static int badFactorial(int n) {  
    return n * badFactorial(n - 1);    // no base case → infinite  
    recursion!  
}  
// Result: StackOverflowError
```

Always define a base case BEFORE the recursive call. Otherwise the recursion never stops and the stack overflows.

Mistake 2: Base Case Never Reached

```
public static int badSum(int n) {  
    if (n == 0) return 0;  
    return n + badSum(n + 1);    // n GROWS — never reaches 0!  
}
```

Each recursive call MUST move toward the base case. If you accidentally increase *n* instead of decreasing it, you'll never stop.

Mistake 3: Returning the Wrong Thing

```
public static int wrongFactorial(int n) {  
    if (n <= 1) return 1;  
    factorial(n - 1);    // forgot to return!  
    return n;            // wrong — always returns n  
}
```

Mistake 4: Modifying Global State Carelessly

Recursive functions that update a shared variable (like a counter) can be hard to debug. Prefer passing state through parameters or returning it explicitly.

Mistake 5: Recursing Too Deep

```
public static int sum(int n) {  
    if (n == 0) return 0;  
    return n + sum(n - 1);  
}
```

```
sum(1000000); // StackOverflowError on most JVMs
```

Java's default stack is ~512 KB. Deep recursion (10,000+ levels) often crashes. Convert to iteration for very deep work.

Mistake 6: Doing Work Twice (Exponential Blow-up)

Like naive fibonacci — recomputing the same subproblem many times. Solution: memoization (caching results) or convert to iterative DP.

13. When to Use Recursion

Recursion is the natural choice for:

- Tree traversals (we'll see in tree chapter — in-order, pre-order, post-order)
- Graph traversal (DFS)
- Divide and conquer (merge sort, quick sort, binary search)
- Backtracking (N-Queens, Sudoku solver, permutations, combinations)
- Dynamic programming (top-down with memoization)
- Problems where the structure is self-similar (Tower of Hanoi, fractals)

Avoid recursion when:

- Simple linear traversal (just use a loop)
- Very deep recursion likely (StackOverflowError risk)
- Performance is critical and iteration is just as clear

14. Quick Reference

Concept	Meaning
Base case	The simplest input where you return directly without recursing
Recursive case	The case where you call the function on smaller input
Call stack	Memory area holding active function call frames
StackOverflowError	When recursion goes too deep and exhausts stack memory
Tail recursion	Recursive call is the LAST operation (no Java optimization)
Memoization	Caching results to avoid redoing the same work
Recursion tree	Visualizing all recursive calls as a tree to analyze complexity
Linear recursion	One recursive call per step (factorial, sum)
Multiple recursion	More than one recursive call per step (fibonacci, tree DFS)

15. Practice Problems

Try each one. Some are warm-ups; later ones get tricky.

Warm-ups

4. Write factorial(n) recursively. Then write it iteratively. Compare.
5. Compute the sum of 1 to N recursively.
6. Compute the sum of digits of a number recursively.
7. Compute a^b without using Math.pow — recursively.
8. Print numbers 1 to N in ASCENDING order recursively, without a loop.
9. Print numbers N to 1 in DESCENDING order recursively, without a loop.
10. Find GCD of two numbers using Euclid's algorithm recursively.

Arrays & Strings

11. Sum all elements of an array recursively.
12. Find the maximum element of an array recursively.
13. Implement linear search recursively.
14. Check if an array is sorted (ascending) recursively.
15. Reverse a string using recursion.
16. Check if a string is a palindrome recursively.
17. Count the number of vowels in a string recursively.
18. Remove all occurrences of a character from a string recursively.

Harder (Interview-style)

19. Compute the Nth Fibonacci number — first naively, then with memoization.
20. Print all subsets of an array (Power set, classic backtracking).
21. Print all permutations of a string.
22. Tower of Hanoi — move N disks from peg A to peg C, using peg B.
23. Count the number of paths from top-left to bottom-right in a grid (only moving right or down).
24. Implement binary search recursively on a sorted array.
25. Generate all balanced parentheses of N pairs ("((()))", "(()())", etc.).
26. Given a number, generate all possible decodings into letters ($1 \rightarrow A$, $2 \rightarrow B$, ..., $26 \rightarrow Z$).

16. Final Takeaway

Recursion is not a trick — it's a different MODE of thinking. Instead of "do this then that then that," you say: "the answer to a big problem is some combination of answers to smaller versions of itself." Once that mode clicks, you'll see recursion everywhere — in math, in data structures, in algorithms.

Don't try to TRACE every recursive call in your head — that path leads to confusion. Instead, TRUST the recursion: define the base case, define how the smaller answer combines into the bigger one, and let the function figure out the rest. That mindset shift is the entire skill.

Key Takeaway: Every recursive function = base case + recursive case + progress toward base case. Trust the function (don't trace it manually). Recognize naive recursion's exponential traps (fibonacci). For interviews, master factorial, fibonacci, array sum, palindrome, and binary search — those five problems cover 80% of recursion fundamentals.

17. What's Next?

Recursion is the gateway to most advanced DSA topics. Natural next steps:

- Backtracking — N-Queens, Sudoku, permutations, combinations
- Trees — binary trees, BSTs, traversals (all recursive!)
- Dynamic Programming — memoization, then tabulation
- Divide & Conquer — Merge Sort, Quick Sort, Binary Search
- Graphs — DFS uses recursion, BFS uses a queue
- Linked Lists — many list operations have elegant recursive solutions
- String algorithms — recursive substring problems, regex matching